



INSTITUTO TECNOLÓGICO SUPERIOR DE LERDO

MATERIA: Administración y Organización de
Datos

NO. UNIDAD: 2

Actividad de Evaluación Corte 1 (data-bridge)

ALUMNOS:

Alexander Adonai Molina Rodriguez - 242310131

Jahaziel Isai Fraire Olivares - 242310878

Jesus Emmanuel Arreola Lopez - 242310336

Jose Angel Torres Cortez - 242310897

DOCENTE:

Jesus Salas Marin

Carrera: Ing. Informática

Semestre: 4°

Lugar: Lerdo, Durango

Fecha de entrega: 06/03/2026

DEFINICIÓN Y PLANTEAMIENTO DEL PROBLEMA

En muchos restaurantes pequeños o negocios de comida rápida, el registro de las órdenes se realiza de manera manual o mediante sistemas muy simples. Esto puede provocar problemas como pérdida de información, dificultad para encontrar órdenes específicas, errores al registrar datos o retrasos en la consulta de pedidos. Cuando el volumen de órdenes aumenta, se vuelve más complicado llevar un control organizado de la información, especialmente cuando se necesita buscar datos específicos como el número de orden, el platillo solicitado, el mesero que atendió o la mesa donde se realizó el pedido.

El problema principal consiste en la falta de un sistema sencillo que permita almacenar y consultar las órdenes del restaurante de manera rápida y organizada. Sin un sistema adecuado, el personal puede tardar más tiempo en localizar información, lo que afecta la eficiencia del servicio y la administración del negocio.

Para solucionar esta situación, se propone desarrollar un sistema de consulta de órdenes que permita registrar, almacenar y buscar información relacionada con los pedidos del restaurante. Este sistema permitirá realizar búsquedas por diferentes criterios, como el identificador de la orden, el tipo de platillo, el mesero que atendió o el número de mesa.

En este proyecto se decidió utilizar archivos planos (.txt) para almacenar la información en lugar de una base de datos tradicional. Esto se debe a que los archivos planos son más sencillos de implementar, no requieren la instalación ni configuración de un sistema gestor de bases de datos y permiten manejar los datos de forma directa mediante programación. Además, para un sistema pequeño o académico como este, los archivos planos son suficientes para guardar y consultar la información sin necesidad de una infraestructura más compleja.

DOCUMENTACIÓN

Generación de datos.

generar.py

Es un programa en Python encargado de generar el archivo **maestro.txt**.

Este script crea datos simulados de órdenes del restaurante y los guarda en el archivo de texto, donde cada línea representa una orden y los datos están separados por el símbolo |.

```
import random
from datetime import datetime
```

Se importan las librerías “random” y “datetime”.

random: se usa para generar valores aleatorios (mesas, meseros, platillos, cantidades, horas).

datetime: está importado por si se quisiera manejar fechas, aunque en este código no se usa directamente.

```
platillos = {
    "Tacos": 25,
    "Gordas": 30,
    "Tamales": 20,
    "Enchiladas": 90,
    "Quesadillas": 35
}
```

Define los platillos disponibles y su precio unitario.

Clave = nombre del platillo.

Valor = precio unitario.

```
meseros = ["Jahaziel", "Zenqu", "Alex", "Torres"]
```

Lista de nombres de meseros que se asignan aleatoriamente a las órdenes.

```
with open("maestro.txt", "w", encoding="utf-8") as archivo:
```

Abre (o crea) el archivo **maestro.txt** en modo escritura ("w").

Si ya existía, lo sobrescribe.

Se asegura que se guarde en UTF-8.

```
archivo.write("ID|Mesa|Mesero|Platillo|Cantidad|PrecioUnitario|Total|Hora\n")
```

Escribe la primera línea del archivo con los nombres de las columnas, separadas por |.

```
for i in range(1, 301):
```

Repite 300 veces para crear las órdenes con IDs del 001 al 300.

```
id_orden = str(i).zfill(3)
```

Convierte el número `i` en string y lo rellena con ceros a la izquierda (001, 002, ...).

```
mesa = random.randint(1, 15)
mesero = random.choice(meseros)
platillo = random.choice(list(platillos.keys()))
precio = platillos[platillo]
cantidad = random.randint(1, 5)
```

Se generan datos al azar, como lo son el de la mesa o el precio.

```
total = precio * cantidad
```

Calcula el total de la orden.

```
hora = random.randint(8, 21)
minuto = random.randint(0, 59)
hora_formato = f"{hora:02d}:{minuto:02d}"
```

Genera una hora aleatoria entre las 08:00 y las 21:59.

`:02d` asegura que siempre tenga dos dígitos (ej. 08:05).

```
línea = f"{id_orden}|{mesa}|{mesero}|{platillo}|{cantidad}|{precio}|{total}|{hora_formato}\n"
```

Une todos los datos en una sola línea, separados por |, y añade salto de línea al final.

```
archivo.write(línea)
```

Escribe la línea en **maestro.txt**.

```
print("Archivo maestro.txt generado correctamente con 300 órdenes.")
```

Muestra en consola un mensaje confirmando que el archivo se creó con éxito.

Interfaz principal

index.php

index.php es la puerta de entrada al sistema:

Presenta el formulario de búsqueda.

Permite filtrar órdenes o mostrar todas.

Envía la información a los archivos PHP que procesan y muestran resultados. Es la parte más visual y didáctica del proyecto, donde se integran

imágenes y estilos.

```
<!--
=====
FORMULARIO DE BÚSQUEDA
Envía los datos al archivo procesar.php
=====
-->

<form action="procesar.php" method="POST" onsubmit="return validarBusqueda()">

<!-- Campo para buscar por ID -->
<label> ID Buscar por ID:</label>
<input type="text" name="id" placeholder="Ejemplo: 005">

<!-- Lista para buscar por tipo de platillo -->
<label> 🍽️ Buscar por Platillo:</label>
<select name="platillo">
<option value="">-- Seleccionar --</option>
<option value="Tacos">Tacos</option>
<option value="Gordas">Gordas</option>
<option value="Tamales">Tamales</option>
<option value="Enchiladas">Enchiladas</option>
<option value="Quesadillas">Quesadillas</option>
</select>

<!-- Lista para buscar por mesero -->
<label> 👤 Buscar por Mesero:</label>
<select name="mesero">
<option value="">-- Seleccionar --</option>
<option value="Jahaziel">Jahaziel</option>
<option value="Zenqu">Zenqu</option>
<option value="Alex">Alex</option>
<option value="Torres">Torres</option>
</select>

<!-- Campo para buscar por número de mesa -->
<label> 🪴 Buscar por Mesa:</label>
<input type="number" name="mesa" placeholder="1 - 15" min="1" max="15">

<!-- Botón para realizar la búsqueda -->
<button type="submit"> 🔍 Buscar Orden</button>

</form>

<!--
=====
FORMULARIO SECUNDARIO
Permite mostrar todas las órdenes registradas
=====
-->

<form action="mostrar_todas.php" method="POST">
<button type="submit" class="secundario">
📄 Mostrar Todas las Órdenes
</button>
</form>

<!-- Descripción pequeña del sistema -->
<p class="descripcion">
Sistema de consulta de órdenes del restaurante.
Los datos se almacenan y procesan utilizando archivos planos.
</p>

</div>

</body>
</html>
```

Procesamiento de búsquedas

procesar.php

Es el módulo encargado de procesar la búsqueda realizada por el usuario.

Recibe los datos enviados desde **index.php**, abre el archivo **maestro.txt** donde se encuentran todas las órdenes del restaurante y compara cada registro con los criterios ingresados. Las órdenes que coinciden con la búsqueda se guardan en el archivo **filtrado.txt**, que posteriormente será utilizado para mostrar los resultados.

```
$id_buscar = $_POST['id'] ?? "";  
$platillo_buscar = $_POST['platillo'] ?? "";  
$mesa_buscar = $_POST['mesa'] ?? "";  
$mesero_buscar = $_POST['mesero'] ?? "";
```

Captura los datos enviados por el formulario (**index.php**) usando método POST.

El operador **?? ""** asegura que si el campo no existe, se asigna cadena vacía en lugar de dar error.

```
if ($id_buscar == "" && $platillo_buscar == "" && $mesa_buscar == "" &&  
$mesero_buscar == "") {  
  
    echo "Debes ingresar al menos un criterio de búsqueda."  
  
    echo "<br><a href='index.php'>Volver</a>";  
  
    exit();  
}
```

Validación en servidor: si todos los campos están vacíos, muestra mensaje de error y detiene la ejecución.

Refuerza la validación que también se puede hacer con JavaScript en el cliente.

```
$maestro = fopen("maestro.txt", "r");
```

Abre el archivo **maestro.txt** en modo lectura.

Este archivo contiene todas las órdenes generadas por **generar.py**.

```
$filtrado = fopen("filtrado.txt", "w");
```

Crea o abre **filtrado.txt** en modo escritura.

Aquí se guardarán las órdenes que cumplan con los criterios de búsqueda.

```
$encabezado = fgets($maestro);  
  
fwrite($filtrado, $encabezado);
```

Lee la primera línea de **maestro.txt** (el encabezado de columnas) y la escribe en **filtrado.txt** para mantener la estructura.

```
while (($linea = fgets($maestro)) !== false) {
```

Recorre el archivo **maestro.txt** línea por línea hasta llegar al final.

```
$datos = explode("|", trim($linea));
```

Divide cada línea en partes usando **|** como separador.

trim() elimina espacios o saltos de línea sobrantes.

El resultado es un arreglo con los campos de la orden.

```
$id = $datos[0];  
  
$mesa = $datos[1];  
  
$mesero = $datos[2];  
  
$platillo = $datos[3];
```

Asigna cada campo a una variable para trabajar más fácilmente.

```
$coincide = true;
```

Variable bandera que indica si la línea cumple con los criterios de búsqueda.

```
if ($id_buscar != "" && $id != $id_buscar) {  
  
    $coincide = false;  
  
}
```

Si el usuario ingresó un ID y no coincide con el de la línea, se marca como no válido.

```
if ($platillo_buscar != "" && $platillo != $platillo_buscar) {  
  
    $coincide = false;}
```

Si el usuario seleccionó un platillo y no coincide, se descarta.


```
if ($mesa_buscar != "" && $mesa != $mesa_buscar) {  
  
    $coincide = false;  
  
}
```

Si el usuario ingresó número de mesa y no coincide, se descarta.

```
if ($mesero_buscar != "" && strtolower($mesero) !=  
strtolower($mesero_buscar)) {  
  
    $coincide = false;  
  
}
```

Si el usuario seleccionó mesero y no coincide (comparación sin importar mayúsculas/minúsculas), se descarta.

```
if ($coincide) {  
  
    fwrite($filtrado, $linea);  
  
}
```

Si la línea cumple con todos los criterios, se escribe en **filtrado.txt**.

```
fclose($maestro);  
  
fclose($filtrado);
```

Cierra ambos archivos para liberar recursos.

```
header("Location: resultados.php");  
  
exit();
```

Redirige automáticamente al archivo **resultados.php** para mostrar los resultados filtrados.

exit() asegura que no se ejecute nada más después de la redirección.

mostrar_todas.php

Este módulo permite visualizar todas las órdenes registradas en el sistema.

Lee directamente el archivo **maestro.txt** y muestra todos los registros sin aplicar ningún filtro, generando una tabla con toda la información disponible.

Apertura y lectura del archivo maestro:

```
$archivo = "maestro.txt";  
$lineas = file($archivo, FILE_IGNORE_NEW_LINES | FILE_SKIP_EMPTY_LINES);
```

Se utiliza la función **file()** de PHP, que es la más eficiente para este caso, ya que lee todo el archivo y lo convierte automáticamente en un arreglo, array, donde cada índice corresponde a una línea del documento, las banderas aseguran que no se procesen saltos de línea innecesarios o líneas vacías que puedan corromper la tabla.

Extracción y limpieza del encabezado:

```
$encabezado = explode("|", array_shift($lineas));
```

El **array_shift** extrae la primera posición del arreglo (la fila de títulos: ID, Mesa, etc.). Inmediatamente, se aplica **explode("|")** para romper esa cadena de texto cada vez que encuentre un pipe, convirtiéndola en un pequeño arreglo de nombres de columnas.

Generación de la estructura de la Tabla (HTML):

```
<table border="1">  
  <thead>  
    <tr>  
      <?php foreach ($encabezado as $columna): ?>  
        <th><?php echo $columna; ?></th>  
      <?php endforeach; ?>  
    </tr>  
  </thead>
```

Se inicia la etiqueta **<table>**, mediante un ciclo **foreach**, el código recorre el arreglo del encabezado generado en el paso anterior para crear dinámicamente las celdas de título (**<th>**). Esto permite que, si en el futuro cambias el orden de las columnas en **generar.py**, la tabla se adapte automáticamente.

Procesamiento y despliegue de los 300 registros:

```
<tbody>
    <?php foreach ($lineas as $linea):
        $datos = explode("|", $linea);
    ?>
    <tr>
        <?php foreach ($datos as $dato): ?>
            <td><?php echo $dato; ?></td>
        <?php endforeach; ?>
    </tr>
    <?php endforeach; ?>
</tbody>
</table>
```

Primer foreach: Recorre el resto de las líneas del archivo, las órdenes simuladas.

explode("|", \$linea): Por cada línea, separa los datos (ID, Mesero, Platillo, etc.) basándose en el delimitador.

Segundo foreach: Recorre cada dato individual de esa línea para insertarlo en una celda estándar (<td>).

El resultado es una malla de datos perfectamente alineada con los encabezados.

Visualización de resultados

resultados.php

Es el módulo que muestra los resultados de la búsqueda al usuario.

Este archivo lee el contenido de **filtrado.txt**, separa cada dato usando el delimitador | y genera una tabla dinámica en HTML donde se muestran las órdenes encontradas dentro del sistema.

```
if (file_exists("filtrado.txt")) {
```

En este bloque el sistema verifica si el archivo **filtrado.txt** existe dentro del servidor.

Esto es importante porque el programa depende de ese archivo para mostrar los resultados de las órdenes filtradas.

Si el archivo no existe, el sistema evitará intentar leerlo, lo que previene errores durante la ejecución.

```
$lineas = file("filtrado.txt");
```

Aquí el programa lee todo el contenido del archivo **filtrado.txt**.

La función **file()** carga cada línea del archivo dentro de un arreglo llamado **\$lineas**.

```
if (count($lineas) <= 1) {  
    echo "<p class='resultado error'>✗ Orden no encontrada.</p>";
```

En este bloque el sistema revisa cuántas líneas tiene el archivo.

El archivo siempre contiene al menos una línea de encabezado, donde aparecen los nombres de las columnas.

Si el archivo tiene solo una línea, significa que no se encontró ninguna orden en la búsqueda.

En ese caso el sistema muestra el mensaje:

“Orden no encontrada”

indicando que no existen registros coincidentes.

```
$total = count($lineas) - 1;  
  
echo "<p class='resultado exito'>✅ Se encontraron $total orden(es).</p>";
```

Aquí se calcula cuántas órdenes fueron encontradas realmente.

Se resta 1 porque la primera línea del archivo corresponde al encabezado.

Después el sistema muestra un mensaje indicando la cantidad de órdenes encontradas.

```
echo "<table>";
```

En este bloque se inicia la estructura de una tabla HTML.

Esta tabla servirá para mostrar los resultados organizados en columnas, facilitando la lectura de los datos.

```
foreach ($lineas as $i => $linea){
```

Aquí se utiliza un ciclo **foreach** para recorrer todas las líneas del archivo.

El ciclo obtiene:

- **\$i** → el número de la línea
- **\$linea** → el contenido de la línea

Esto permite procesar cada orden de manera individual.

```
$datos = explode("|", trim($linea));
```

En este bloque el sistema separa los datos de cada línea.

Cada línea del archivo está organizada con el delimitador |.

Por ejemplo:

005|3|Carlos|Tacos|2|25|50|12:30

La función **explode()** divide esa línea en un arreglo con cada dato separado:

ID

Mesa

Mesero

Platillo

Cantidad

Precio

Total

Hora

Esto permite mostrar cada valor en su propia columna dentro de la tabla.

```

if($i == 0){
    echo "<tr>";

    foreach($datos as $dato){
        echo "<th>$dato</th>";
    }

    echo "</tr>";
}

```

Este bloque detecta si la línea actual es la primera del archivo.

La primera línea corresponde al encabezado, que contiene los nombres de las columnas.

Por eso el sistema crea una fila de tabla usando etiquetas **<th>**, que se utilizan para encabezados.

Esto genera los títulos de cada columna de la tabla.

```

else{
    echo "<tr>";

    foreach($datos as $dato){
        echo "<td>$dato</td>";
    }

    echo "</tr>";
}

```

En este bloque se procesan las líneas restantes del archivo, que representan las órdenes del restaurante.

Cada dato de la orden se coloca dentro de una celda de la tabla usando la etiqueta **<td>**.

Esto permite mostrar los datos organizados por columnas dentro de la tabla.

Cada iteración del ciclo genera una nueva fila con la información de una orden.

Descarga de resultados

Gestión de Descarga de Archivos (descargar_txt.php):

Este archivo permite que el usuario obtenga el resultado de sus búsquedas, almacenado en **filtrado.txt**, como un archivo físico descargable.

```
$archivo = "filtrado.txt";
```

```
/*  
Verifica que el archivo exista antes de intentar enviarlo.  
*/  
if (file_exists($archivo)) {
```

```
/*  
header():  
Envía cabeceras HTTP para forzar descarga.  
*/  
  
header("Content-Type: text/plain");  
header("Content-Disposition: attachment; filename=archivo.txt");  
header("Content-Length: " . filesize($archivo));
```

```
/*  
readfile():  
Envía el contenido del archivo directamente al navegador.  
*/  
readfile($archivo);  
exit();  
  
} else {  
    echo "El archivo no existe."  
}
```

Validación de existencia: Antes de procesar nada, **file_exists(\$archivo)** evita errores de servidor si el usuario intenta descargar sin haber realizado un filtrado previo.

Configuración de Cabeceras (MIME Types):

Content-Type: text/plain: Indica al navegador que el contenido es texto puro.

Content-Disposition: attachment; filename=archivo.txt: Esta es la línea clave, porque el parámetro **attachment** le dice al navegador que no debe intentar mostrar el archivo, sino abrir el cuadro de diálogo de "Guardar como".

Content-Length: Informa al navegador el tamaño del archivo para que pueda mostrar una barra de progreso de descarga si fuera necesario.

Transmisión: **readfile(\$archivo)** lee el contenido del disco y lo vuelca al flujo de salida (output buffer), completando la transferencia de datos hacia el dispositivo del usuario, el **exit()** final previene que se envíe código adicional accidentalmente.

estilos.css

estilos.css es el maquillaje del sistema:

Hace que el formulario sea atractivo y fácil de usar.

Diferencia botones y campos para que sean claros.

Da formato a las tablas de resultados.

Mejora la presentación con imágenes y simetría.

```

19  body {
20
21      /*
22      Define la tipografía principal del sistema.
23      Arial es una fuente limpia y profesional.
24      */
25      font-family: Arial, sans-serif;
26
27      /*
28      Color de fondo general de la página.
29      Se utiliza un tono beige para simular
30      ambiente cálido tipo restaurante mexicano.
31      */
32      background-color: #f4e1c1;
33
34      /*
35      Elimina márgenes externos predeterminados del navegador.
36      */
37      margin: 0;
38
39      /*
40      Elimina relleno interno predeterminado.
41      */
42      padding: 0;
43  }
44
45
46  /*
47  -----
48  CONTENEDOR PRINCIPAL
49  -----
50  Caja central donde se muestra el contenido.
51  */
52  .contenedor {
53
54      /*
55      Define el ancho del contenedor al 50%
56      del ancho total de la ventana.
57      */
58      width: 50%;
59
60      /*
61      Centra el contenedor horizontalmente.
62      50px arriba y abajo, auto a los lados.
63      */
64      margin: 50px auto;
65
66      /*
67      Fondo blanco para contraste con el body.
68      */
69      background-color: white;
70
71      /*
72      Espacio interno entre el contenido
73      y los bordes del contenedor.
74      */
75      padding: 30px;
76

```

maestro.txt

Es un archivo plano de texto que actúa como la base de datos inicial del sistema.

Contiene todas las órdenes simuladas que se generan con generar.py.

Cada línea representa una orden y está separada por el delimitador |.

```
1 ID|Mesa|Mesero|Platillo|Cantidad|PrecioUnitario|Total|Hora
2 001|13|Alex|Gordas|2|30|60|09:55
3 002|1|Alex|Quesadillas|1|35|35|12:07
4 003|10|Zenqu|Gordas|4|30|120|15:30
5 004|1|Torres|Gordas|5|30|150|18:26
6 005|11|Zenqu|Quesadillas|1|35|35|09:53
7 006|7|Alex|Tamales|5|20|100|13:07
8 007|14|Alex|Tamales|1|20|20|20:07
9 008|7|Torres|Tacos|1|25|25|12:41
10 009|7|Jahaziel|Enchiladas|2|90|180|16:36
11 010|5|Torres|Gordas|5|30|150|20:07
12 011|4|Jahaziel|Gordas|2|30|60|11:15
13 012|5|Torres|Quesadillas|5|35|175|20:55
14 013|3|Alex|Tamales|5|20|100|12:25
15 014|10|Alex|Gordas|4|30|120|17:10
16 015|6|Jahaziel|Enchiladas|3|90|270|19:50
```

filtrado.txt

Es un archivo plano de texto que funciona como archivo temporal de resultados del sistema.

Se genera automáticamente cada vez que el usuario realiza una búsqueda o solicita mostrar todas las órdenes.

Contiene únicamente las órdenes que coinciden con los criterios de búsqueda ingresados por el usuario (ID, platillo, mesero o mesa).

Las líneas se copian desde maestro.txt, manteniendo la misma estructura y el mismo delimitador |, por lo que cada línea representa una orden filtrada.

Este archivo permite que el sistema separe el proceso de filtrado del proceso de visualización, ya que primero se guardan los resultados y después resultados.php los lee para generar la tabla dinámica en la página web o permitir su descarga como archivo **.txt**.

```
1 ID|Mesa|Mesero|Platillo|Cantidad|PrecioUnitario|Total|Hora
2 009|7|Jahaziel|Enchiladas|2|90|180|16:36
3 011|4|Jahaziel|Gordas|2|30|60|11:15
4 015|6|Jahaziel|Enchiladas|3|90|270|19:50
5 016|2|Jahaziel|Quesadillas|3|35|105|16:10
6 027|11|Jahaziel|Enchiladas|2|90|180|21:41
7 038|13|Jahaziel|Enchiladas|2|90|180|14:13
8 046|13|Jahaziel|Enchiladas|1|90|90|08:44
9 047|7|Jahaziel|Tacos|4|25|100|16:28
10 048|8|Jahaziel|Tacos|5|25|125|20:34
11 059|8|Jahaziel|Tamales|2|20|40|14:48
12 060|1|Jahaziel|Gordas|1|30|30|21:11
13 062|2|Jahaziel|Enchiladas|5|90|450|13:46
14 071|8|Jahaziel|Tamales|3|20|60|13:52
15 072|5|Jahaziel|Gordas|2|30|60|17:10
16 073|6|Jahaziel|Quesadillas|5|35|175|16:53
17 079|2|Jahaziel|Tacos|2|25|50|18:09
```

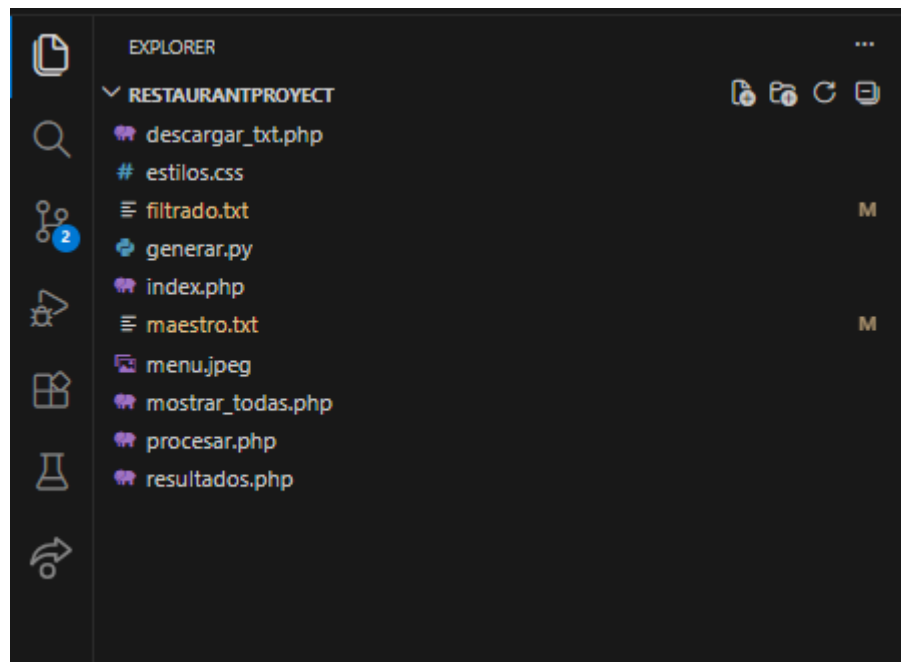
menu.jpeg

Es una imagen utilizada dentro de la interfaz del sistema.

Su función es únicamente visual, ayudando a representar el contexto del restaurante dentro de la página principal (index.php).



El explorador en VS Code quedo asi:



10 archivos

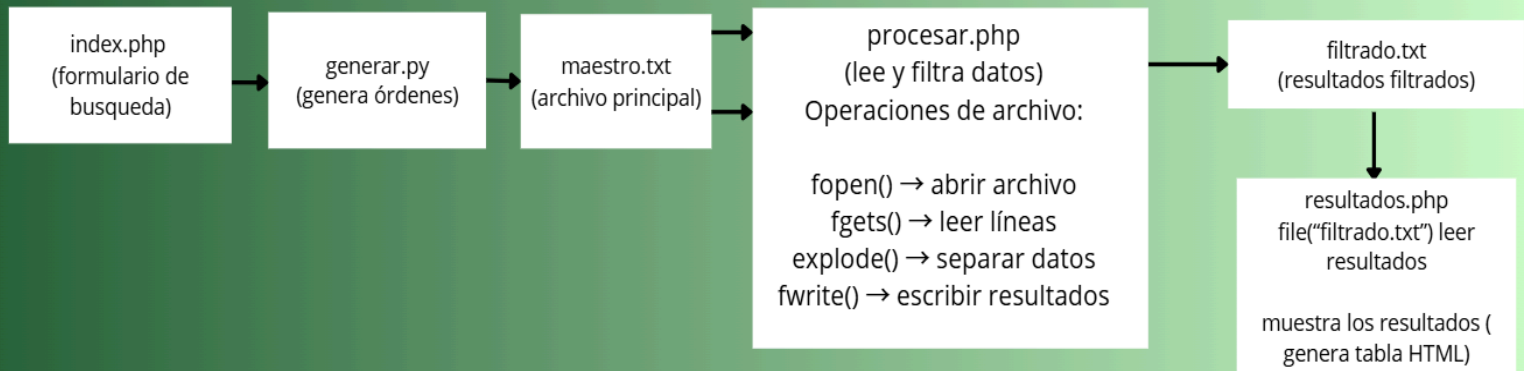
Maqueta

Archivo maestro.txt

El sistema almacena las órdenes del restaurante en un archivo plano llamado "maestro.txt".
Cada registro representa una orden realizada en el restaurante.
Los datos se organizan en columnas separadas por el símbolo "|".
Cada línea contiene la siguiente información:

```
ID|Mesa|Mesero|Platillo|Cantidad|PrecioUnitario|Total|Hora
001|1|Jahaziel|Enchiladas|2|90|180|18:26
002|12|Torres|Gordas|3|30|90|19:16
003|4|Torres|Quesadillas|5|35|175|11:37
004|6|Torres|Tamales|3|20|60|11:15
005|7|Alex|Quesadillas|5|35|175|13:44
006|5|Torres|Tacos|2|25|50|14:12
007|4|Jahaziel|Tamales|4|20|80|17:55
```

Operaciones del sistema



La estética final de la aplicación.

Buscar Órdenes

Consulta las órdenes registradas en el sistema del restaurante.

Filtros de búsqueda

ID Buscar por ID:

Ejemplo: 005

Buscar por Platillo:

-- Seleccionar --

Buscar por Mesero:

-- Seleccionar --

Buscar por Mesa:

1 - 15

Resultados de Órdenes

✅ Se encontraron 300 orden(es).

ID	Mesa	Mesero	Platillo	Cantidad	PrecioUnitario	Total	Hora
001	8	Alex	Quesadillas	2	35	70	20:35
002	10	Jahaziel	Gordas	3	30	90	21:22
003	4	Torres	Tamales	3	20	60	18:29
004	6	Alex	Tamales	1	20	20	20:30
005	1	Alex	Enchiladas	2	90	180	15:41
006	7	Torres	Enchiladas	5	90	450	13:43
007	10	Torres	Tamales	4	20	80	13:13
008	4	Alex	Tamales	5	20	100	20:29

La interfaz del sistema fue desarrollada en HTML, CSS y PHP.
Permite al usuario buscar órdenes mediante distintos criterios como ID, platillo, mesero o número de mesa.

Los resultados de la búsqueda se muestran en una tabla dinámica generada